

Configuratore Borse 2D

Documento Tecnico Integrale di Progetto

Architettura · Database · Engine di Rendering · Pipeline Asset · Edge Functions ·
Sicurezza · Roadmap

Versione documento	1.0
Data redazione	Maggio 2026
Destinatari	Ingegneri software, lead tecnici, art directors
Stack	React 18 + Vite + TypeScript · Lovable Cloud (Supabase) · Canvas 2D HTML5
Pagine	25+
Classificazione	Documento tecnico riservato

Indice

Sezione	Pag.
1. Introduzione e visione di insieme	3
2. Architettura generale del sistema	4
3. Stack tecnologico e linguaggi	5
4. Modello dati (Database relazionale)	6
5. Row Level Security e policy	9
6. Edge Functions (logica server-side)	10
7. Storage e gestione dei file immagine	12
8. Engine di rendering 2D Canvas	13
9. Pipeline di composizione livelli (Z-index)	15
10. Gestione tessuti e texture seamless	17
11. Gestione manici, path e strisce longitudinali	18
12. Maschere, feathering e overlay	20
13. Frontend e interfaccia utente	21
14. Backend amministrativo (Admin Panel)	22
15. Pipeline di import asset (ZIP canonico)	24
16. Flusso utente (6 step) e integrazione Shopify	25
17. Memorizzazione utenti e gestione sessione	26
18. Funzionalità deboli, debiti tecnici, rischi produzione	27
19. Performance, colli di bottiglia e ottimizzazioni	28
20. Stato dell'arte grafico e proposte di miglioramento	29
21. Roadmap di evoluzione e raccomandazioni finali	30
Appendice A — Glossario tecnico	31
Appendice B — Diagramma ER sintetico	32

1. Introduzione e visione di insieme

Il progetto è un **configuratore 2D di borse personalizzabili** sviluppato come Single Page Application React+Vite, disaccoppiato dal canale e-commerce (Shopify) e backed da un'infrastruttura Lovable Cloud (istanza gestita di Supabase). Il dominio applicativo è artigianale-luxury: l'utente finale sceglie un modello di borsa e ne personalizza le componenti (tessuto, colore, manici, ricamo) ottenendo un *preview fotorealistico* in tempo reale, generato lato browser tramite composizione di livelli su Canvas HTML5 2D.

Il sistema è progettato come **configuratore agnostico rispetto al carrello**: lo stato configurazione viene serializzato in *Line Item Properties* Shopify al momento del checkout, mentre il database custom mantiene il catalogo di modelli, tessuti, manici, ricami, regole di compatibilità e prezzi. Questa separazione consente di evolvere il configuratore senza dipendenze strette dal frontend Shopify.

La filosofia di rendering è **compositiva e non procedurale**: il prodotto finale è la sovrapposizione di N livelli PNG (base, maschere zona, texture tessuto, overlay ombre, overlay luci, dettagli hardware, manici procedurali, ricamo posizionato), ciascuno con il proprio z-index, blend mode e opacità. Le maschere bianco-su-trasparente con feathering 1–3px definiscono i boundary delle zone tessuto, mentre gli overlay multiply/screen restituiscono profondità e realismo materico. Tutti gli asset di una stessa vista devono avere *identica dimensione canvas e allineamento pixel-perfect*, vincolo cardine dell'intera pipeline grafica.

Il documento che segue è una guida tecnica esaustiva: dopo una panoramica architetturale, analizza in dettaglio ogni componente — schema dati, RLS, edge functions, storage, engine di rendering, pipeline di import, frontend, backend admin, integrazione Shopify — e si conclude con una valutazione critica delle aree fragili, dei colli di bottiglia di performance e con una roadmap di miglioramento per portare il prodotto a un livello qualitativo Awwwards-grade. L'obiettivo dichiarato è permettere a un ingegnere software competente di **ricostruire integralmente il progetto da zero** partendo da questo solo documento.

1.1 Obiettivi di prodotto

- Offrire al cliente finale un'esperienza di personalizzazione borse fluida, in tempo reale, accessibile da desktop e mobile.
- Garantire al brand un controllo totale sul catalogo (modelli, tessuti, manici, ricami) tramite un pannello amministrativo interno.
- Disaccoppiare il configuratore dal motore e-commerce per consentire deploy indipendenti, A/B test e refactoring grafico senza rischi sulla pipeline ordini.
- Mantenere prezzi deterministici e tracciabili: il prezzo finale dipende esclusivamente da modello + tessuto, mentre manici, colori e ricami sono variazioni stilistiche a costo zero.
- Persistere le configurazioni in formato leggibile (sessione → riga ordine Shopify) per consentire ri-rendering a posteriori, gestione resi e analisi marketing.

1.2 Stato attuale (snapshot)

Alla data di redazione il sistema è in stato **pre-produzione avanzato**: il backend è funzionante, l'engine di rendering compone correttamente i livelli, il pannello admin permette di caricare borse complete via ZIP, l'integrazione AI per la generazione di texture seamless è attiva via Lovable AI Gateway. Tuttavia il *risultato grafico finale è ancora limitato*: gli asset attuali producono un'estetica che ricorda più un mockup

tecnico che un rendering fotorealistico definitivo. Il flusso di onboarding asset (ZIP, naming canonico, normalizzazione alias) è funzionale ma poco user-friendly per art director non tecnici. La sezione 18–20 di questo documento entra nel dettaglio di queste criticità e propone interventi mirati.

2. Architettura generale del sistema

L'architettura segue il pattern **SPA + BaaS** con tre piani logici nettamente separati:

Piano 1 — Client (browser)

React 18 + Vite 5 in TypeScript 5. Il client gestisce *tutto* il rendering grafico (nessuna immagine viene composta server-side), la UI di configurazione, l'autenticazione opzionale, l'interazione con il pannello admin, l'upload diretto a Storage tramite Supabase JS SDK. La componente più rilevante è `src/engine/`, motore di composizione 2D che opera su un singolo `HTMLCanvasElement` con buffer `offscreen` per i passi intermedi.

Piano 2 — Lovable Cloud (Supabase managed)

Backend gestito che fornisce: PostgreSQL 15 con Row Level Security, Storage S3-compatible (due bucket pubblici: `configurator-assets` per il catalogo e `admin-assets` per i ZIP di import), Auth (email+password, opzionale Google OAuth), Edge Functions (Deno runtime, V8 isolate), e l'AI Gateway che fa da proxy verso Gemini/OpenAI senza richiedere API key all'utente. La connessione è stabilita in `src/integrations/supabase/client.ts`, file auto-generato e immutabile.

Piano 3 — Shopify (e-commerce esterno)

Shopify resta il sistema di record per ordini, pagamenti, fulfillment. Il configuratore inietta nel carrello uno SKU prodotto + un blob serializzato di Line Item Properties contenente: `modelSlug`, `fabricSlug`, `fabricColorHex`, `handleSlug`, `handleColorHex`, `embroideryId`, prezzo finale, URL preview, `sessionId`. La consegna è gestita dal flusso Shopify standard.

2.1 Diagramma ad alto livello

Il flusso applicativo è il seguente:

```
[Browser SPA React]
|
|--(REST/Realtime SDK)--> [Lovable Cloud]
| |-- PostgreSQL (catalogo, configurazioni)
| |-- Storage (PNG modelli, texture, ZIP)
| |-- Auth (utenti opzionali)
| |-- Edge Funcs (analyze-handle-geometry,
| enhance-texture,
| seamless-from-photo)
| \-- AI Gateway --> Gemini 3.x / GPT-5
|
|--(Line Item Props)----> [Shopify Cart]
|-- Pagamento
\-- Fulfillment
```

2.2 Principi architetturali

- **Stateless rendering:** il client riceve dati JSON dal DB, scarica gli asset PNG dal Storage, e ricompone localmente. Nessuna immagine pre-renderizzata viene salvata server-side (tranne il preview finale opzionale per Shopify).
- **Idempotenza degli import:** l'upload di un ZIP per un modello+vista esistente esegue `upsert` sui record DB, mai duplicazione. Questo permette di ri-caricare correzioni senza creare ghost record.
- **Decoupling totale dal CMS:** il configuratore non sa nulla di Shopify se non l'ID prodotto. Lato Shopify, il configuratore è un'`iframe/embed` neutrale.

- **Catalogo dichiarativo:** ogni regola (compatibilità, prezzo, layer order) è espressa come riga di tabella, non come codice. L'admin modifica righe, mai files.

3. Stack tecnologico e linguaggi

3.1 Frontend

Il frontend è un'applicazione React 18 SPA buildata con Vite 5. Linguaggio principale: **TypeScript 5** con configurazione strict moderata. Il bundler usa esbuild per il dev server e Rollup per la build di produzione. Il routing è gestito da react-router-dom v6, lo state management è basato su React hooks (useState, useReducer, custom hooks dedicati come useRenderingData) più **@tanstack/react-query v5** per il caching delle query verso Supabase.

3.2 UI Toolkit

Tailwind CSS v3 fornisce le utility CSS, configurate con design tokens semantici HSL (variabili CSS in src/index.css: --background, --foreground, --primary, --accent, --destructive ecc.). Sopra Tailwind è montato **shadcn/ui**, libreria di componenti basata su Radix UI primitives (Dialog, Popover, Select, Slider, Tabs, Toast). Le icone sono fornite da lucide-react. La componentistica admin sfrutta intensivamente Tabs, Dialog, Sheet per modali laterali, Sonner per notifiche toast.

3.3 Backend

Lovable Cloud espone PostgreSQL 15 + estensioni standard (uuid-oss, pgcrypto). Le Edge Functions girano su **Deno runtime** (TypeScript nativo, V8 isolate, cold start ~50ms), deploy automatico al push del file. Lo Storage è S3-compatible con bucket pubblici o privati. L'AI Gateway è un proxy interno che espone endpoint compatibili OpenAI (/v1/chat/completions) verso modelli Gemini, GPT-5 e varianti image, autenticato tramite la variabile d'ambiente LOVABLE_API_KEY.

3.4 Librerie chiave

Libreria	Versione	Ruolo
react / react-dom	18.3	Framework UI
@supabase/supabase-js	2.103	SDK client per DB/Storage/Auth/Edge
@tanstack/react-query	5.83	Caching/sincronizzazione stato server
react-router-dom	6.30	Routing client-side
react-hook-form + zod	7.61 / 3.x	Form validation tipizzata
jszip	3.10	Parser/builder ZIP per import asset
lucide-react	0.462	Icon set SVG tree-shakable
shadcn/ui + Radix	latest	Componenti UI accessibili
tailwindcss	3.4	Utility CSS + design tokens
vitest	latest	Test runner unitari

3.5 Linguaggi e file count

Il codebase si compone di circa 5.500 LOC TypeScript distribuiti in: src/engine/ (~2.000 LOC, motore di rendering), src/lib/ (~2.700 LOC, utility di import asset, manipolazione texture, gestione overlay), src/components/admin/ e src/pages/admin/ (~3.000+ LOC, UI pannello admin), supabase/functions/ (~750 LOC TypeScript Deno, edge functions). Il frontend cliente finale è volutamente più snello (~1.500

LOC) perché delega quasi tutto al motore di rendering.

4. Modello dati (Database relazionale)

Il database è organizzato in **18 tabelle** nel namespace public, modellate per riflettere la gerarchia logica del dominio: modello → vista → zona maschera, modello → tessuti compatibili → colori, modello → manici → geometrie → side parts, e configurazioni utente come tabella di trace. Ogni tabella usa UUID come PK (gen_random_uuid()) e include i timestamp standard created_at e updated_at aggiornato via trigger PL/pgSQL update_updated_at_column().

4.1 Tabelle catalogo

Tabella	Cardinalità	Scopo
bag_models	~10–50	Modello fisico di borsa (City, Bauletto, Tote...). Contiene name, slug, base_price, shopify_pro
bag_views	3–6 per modello	Vista geometrica del modello (front, back, three_quarter). Contiene canvas_width/height, bas
mask_zones	4–10 per vista	Zona pittabile della borsa (fabric_front, fabric_side, fabric_top, fabric_bottom, overlay_zipper
fabrics	~30–100	Tessuto/materiale (es. Neoprene_base, pelle_saffiano). Contiene texture_url (PNG seamless
fabric_colors	~5–10 per fabric	Colore disponibile per un tessuto. Contiene hex (#RRGGBB), thumbnail_url, opzionale deriv
handles	~10–30	Tipo di manico (centrale, laterale, tracolla). Contiene name, slug, category.
handle_geometries	1 per (vista,manico)	Geometria fissa del manico in una vista: mask_url, shadow_url, highlight_url, hardware_url, p
handle_side_parts	0–4 per geometry	Parti laterali del manico (es. fettuccine vicino agli anelli) con propri mask/path/shadow/highlig
handle_pattern_presets	~20–50	Preset di pattern righe procedurali (stripe_count, preset_json contenente l'array di colori/largh
handle_colors	~5–10 per handle	Tinta unita applicabile al manico.
cord_collection	~10–20	Catalogo cordame globale (texture PNG di corde a strisce).
embroideries	~50–200	Catalogo ricami (PNG con alpha, dimensioni libere).
embroidery_placements	1 per vista	Posizione/scala/rotazione default del ricamo per una vista (positionX%, positionY%, maxWid

4.2 Tabelle relazionali e regole

- **compatibility_rules** — Matrice modello↔tessuto e modello↔manico. Stabilisce quali combinazioni sono ammesse. Letta dal frontend per filtrare le opzioni disponibili.
- **cord_handle_compatibility** — Mapping tra cordame e tipo di manico (un manico ammette solo certe corde).
- **pricing_rules** — Tabella che esprime il prezzo finale come funzione di (bag_model_id, fabric_id). Letta in fase di riepilogo.
- **layer_order_rules** — Override del layer order di default per vista specifica (z_index, blend_mode, opacity, isActive per ogni layer_type).
- **configurations** — Sessione di configurazione utente (session_id testuale o user_id se autenticato). Persiste lo snapshot finale (fabric_id, handle_id, embroidery_id, final_price, preview_url, shopify_product_id, shopify_variant_id, status, expires_at).

4.3 Schema esemplificativo (DDL)

```
CREATE TABLE public.mask_zones (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  bag_view_id uuid NOT NULL REFERENCES bag_views(id) ON DELETE CASCADE,
```

```

zone_type text NOT NULL, -- 'fabric_front' | 'overlay_zipper' | ...
zone_category text NOT NULL DEFAULT 'fabric', -- fabric|handle|embroidery|overlay
label text,
mask_image_url text,
local_overlay_url text,
texture_url text, -- override per-zona
z_index int NOT NULL DEFAULT 0,
sort_order int NOT NULL DEFAULT 0,
blend_mode text NOT NULL DEFAULT 'normal',
shading_strength numeric NOT NULL DEFAULT 1.0,
tint_color text, -- hex per overlay zipper
texture_scale numeric NOT NULL DEFAULT 1.0,
texture_offset_x numeric NOT NULL DEFAULT 0,
texture_offset_y numeric NOT NULL DEFAULT 0,
texture_rotation numeric NOT NULL DEFAULT 0,
texture_repeat_mode text NOT NULL DEFAULT 'repeat',
scale_correction_factor numeric NOT NULL DEFAULT 1.0,
created_at timestamptz NOT NULL DEFAULT now(),
updated_at timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX idx_mask_zones_view ON public.mask_zones(bag_view_id);

```

4.4 Trigger di timestamp

```

CREATE OR REPLACE FUNCTION public.update_updated_at_column()
RETURNS trigger LANGUAGE plpgsql
SET search_path = public AS $$
BEGIN
NEW.updated_at = now();
RETURN NEW;
END $$;

```

Il trigger è applicato a ciascuna tabella tramite CREATE TRIGGER ... BEFORE UPDATE FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();. Si noti la clausola SET search_path = public, mitigazione di sicurezza contro l'attacco di search_path injection in funzioni SECURITY DEFINER.

5. Row Level Security e policy

Lo schema attuale espone **58 policy** distribuite su 18 tabelle, organizzate per operazione (SELECT/INSERT/UPDATE/DELETE) per ciascuna tabella. La filosofia attuale è *volutamente permissiva*: tutte le policy ammettono USING (true) e WITH CHECK (true). Questa scelta riflette il fatto che il configuratore è attualmente **aperto al pubblico** sia in lettura (catalogo navigabile da chiunque) sia in scrittura (l'admin opera senza autenticazione abilitata in modalità preview).

■ **Nota:** Questa configurazione è accettabile in fase di prototipo/staging ma deve essere ristretta prima del go-live di produzione. La sezione 18 lista questo punto come rischio di sicurezza prioritario, da sanare introducendo un sistema di ruoli (tabella `user_roles` + funzione `SECURITY DEFINER has_role()`) e policy discriminanti tra utente anonimo (read-only sul catalogo), utente autenticato (CRUD sulle proprie configurations) e utente admin (CRUD pieno).

5.1 Pattern target per la produzione

```
-- 1. Tabella ruoli separata (mai sul profilo)
CREATE TYPE public.app_role AS ENUM ('admin', 'editor', 'user');

CREATE TABLE public.user_roles (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id uuid NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  role app_role NOT NULL,
  UNIQUE(user_id, role)
);
ALTER TABLE public.user_roles ENABLE ROW LEVEL SECURITY;

-- 2. Funzione SECURITY DEFINER (evita ricorsione su user_roles)
CREATE OR REPLACE FUNCTION public.has_role(_user_id uuid, _role app_role)
RETURNS boolean LANGUAGE sql STABLE SECURITY DEFINER
SET search_path = public AS $$
SELECT EXISTS(SELECT 1 FROM public.user_roles
WHERE user_id = _user_id AND role = _role);
$$;

-- 3. Esempio policy ristretta su bag_models
CREATE POLICY "Public read bag_models"
ON public.bag_models FOR SELECT TO public USING (true);

CREATE POLICY "Admin write bag_models"
ON public.bag_models FOR ALL TO authenticated
USING (public.has_role(auth.uid(), 'admin'))
WITH CHECK (public.has_role(auth.uid(), 'admin'));
```

5.2 Policy attuali — sintesi

Tutte le tabelle del catalogo (*bag_models*, *bag_views*, *mask_zones*, *fabrics*, *fabric_colors*, *handles*, *handle_geometries*, *handle_side_parts*, *handle_pattern_presets*, *handle_colors*, *cord_collection*, *layer_order_rules*, *embroideries*, *embroidery_placements*, *compatibility_rules*, *cord_handle_compatibility*, *pricing_rules*) hanno RLS abilitata e policy aperte. La tabella **configurations** è del tutto pubblica anche in lettura: questo permette il flusso "configura → carrello" senza login ma espone storicamente tutte le sessioni. È prevista una migrazione che restringa la SELECT su `session_id` = `current_setting('request.headers')::json->>'x-session-id'` oppure a `auth.uid() = user_id` se l'utente è loggato.

6. Edge Functions (logica server-side)

Le Edge Functions del progetto sono **tre**, tutte scritte in TypeScript per Deno runtime e deployate sotto il path `supabase/functions/<name>/index.ts`. Ogni funzione espone un endpoint HTTP POST raggiungibile via `{SUPABASE_URL}/functions/v1/<name>`. Il flag `verify_jwt` è disabilitato di default per consentire chiamate da utente anonimo durante la configurazione.

6.1 seamless-from-photo

Scopo: estrarre da una fotografia di tessuto reale un *tile seamless* ad alta fedeltà adatto a ripetizione toroidale, da usare come `texture_url` in `fabrics`.

Input: data URL base64 dell'immagine sorgente.

Pipeline: prepara un prompt specializzato per estrazione tessile (real fibers, real weave, lighting flat, edge alignment), invia richiesta multimodale all'AI Gateway sul modello `google/gemini-3.1-flash-image-preview` con `modalities ['image','text']`, intercetta il timeout a 70s server-side.

Output: data URL del tile generato + metadata. Il client poi carica il blob su `configurator-assets`.

```
const aiResp = await fetch('https://ai.gateway.lovable.dev/v1/chat/completions', {
  method: 'POST',
  headers: {
    'Authorization': `Bearer ${Deno.env.get('LOVABLE_API_KEY')}`,
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({
    model: 'google/gemini-3.1-flash-image-preview',
    modalities: ['image', 'text'],
    messages: [{
      role: 'user',
      content: [
        { type: 'text', text: SEAMLESS_PROMPT },
        { type: 'image_url', image_url: { url: imageDataUrl } }
      ]
    }]
  })
});
```

6.2 enhance-texture

Scopo: post-processare una texture esistente per migliorarne contrasto, illuminazione uniforme, eliminazione di artefatti. Usata come step opzionale dopo `seamless-from-photo`. Prompt orientato a "flatten lighting, preserve fiber structure". Stesso modello `image-preview`, stesso pattern di chiamata.

6.3 analyze-handle-geometry

Scopo: dato il PNG di un manico (mask con alpha), analizzare la geometria del manico e proporre una *centerline polilinea* con tangenti per ciascun punto, da salvare in `handle_geometries.path_json`.

Pipeline: la funzione delega a un modello reasoning (`google/gemini-2.5-pro`) la stima dello skeleton, ma il calcolo effettivo della scheletrizzazione resta lato client (vedi sezione 11) tramite algoritmo `nearest-neighbor` sui pixel della maschera. La funzione qui agisce come **helper** per la proposta automatica di segmentazione tangenti.

6.4 Pattern di chiamata dal client

```
const { data, error } = await supabase.functions.invoke('seamless-from-photo', {
  body: { imageUrl },
});
if (error) throw error;
const blob = await (await fetch(data.tileDataUrl)).blob();
const path = `fabrics/${slug}/seamless_${Date.now()}.png`;
await supabase.storage.from('configurator-assets').upload(path, blob);
```

6.5 Sicurezza delle Edge Functions

- Le funzioni sono **pubbliche** (verify_jwt=false) per supportare guest checkout. Questo le rende potenzialmente abusabili per drenare quota AI Gateway.
- Mitigazione raccomandata: rate-limit per IP via Edge KV (Deno KV), oppure verify_jwt=true e un endpoint pubblico solo dopo magic link.
- Le risposte non vengono cached: ogni chiamata genera nuova fattura AI. Per testura ricorrente conviene cache hash-based su Storage.

7. Storage e gestione dei file immagine

Il sistema usa due bucket S3-compatible:

- **configurator-assets** (pubblico) — Tutto il catalogo: PNG modelli (base, overlay shadows/highlights/details), maschere, texture tessuti, manici, ricami, thumbnail. Le URL pubbliche sono direttamente CDN-served e cacheable.
- **admin-assets** (pubblico) — ZIP di import asset, file temporanei, anteprime di lavorazione. È pubblico per semplicità ma andrebbe ristretto.

7.1 Convenzioni di naming

```
configurator-assets/  
bag_models/  
city/  
front/  
base.png  
overlay_shadows.png  
overlay_highlights.png  
overlay_details.png  
masks/  
mask_body_front_main.png  
mask_body_left_side.png  
...  
handle/  
mask_handle_main_full.png  
handle_path.json  
overlay_handle_shadows.png  
...  
fabrics/  
neoprene_base/  
texture.png  
seamless_<timestamp>.png  
embroideries/  
flower_01.png
```

7.2 Vincoli sui file

- PNG 32-bit RGBA, sfondo trasparente. JPEG accettato solo per texture tessuto (no alpha channel necessario).
- Tutti i PNG di una stessa vista (base + tutti gli overlay + tutte le maschere) DEVONO avere identiche dimensioni canvas (es. 1762×1770) e identico allineamento pixel-perfect. La validazione è effettuata in `src/engine/assetValidator.ts` via `validateImageDimensions`.
- Maschere: bianco #FFFFFF su trasparente, feather 1–3px sui bordi, niente hardware metallico (gestito da `overlay_details`).
- Overlay shadows: PNG grayscale con alpha, applicato in blend multiply.
- Overlay highlights: PNG grayscale con alpha, applicato in blend screen, opacity ~0.8.
- Overlay details: PNG con colore, applicato in blend normal sopra tutto (cerniere, fibbie, etichette).

7.3 Upload pattern

Tutti gli upload passano per `src/lib/uploadAsset.ts` che fornisce un'API unificata `uploadAsset(blob, path, bucket?)`. Il caricamento usa `upsert: true` per sostituire file esistenti (idempotenza), `contentType` dedotto dal MIME del blob, e `cacheControl: '3600'` per evitare stale cache CDN durante l'iterazione admin.

8. Engine di rendering 2D Canvas

Il cuore del progetto è `src/engine/`, motore di composizione su `HTMLCanvasElement` 2D che produce il preview borsa in tempo reale (<60ms target frame budget). Esposto via `composeScene()`, accetta un oggetto `RenderScene = view + selection` e disegna su un canvas target.

8.1 Tipi fondamentali

```
interface RenderScene { view: RenderView; selection: RenderSelection; }

interface RenderView {
  id: string;
  viewType: string;
  canvasWidth: number;
  canvasHeight: number;
  baseImageUrl: string | null;
  overlays: ViewOverlay[]; // shadows | highlights | details
  maskZones: RenderMaskZone[]; // zone tessuto/handle/overlay
  layerOrder: RenderLayerOrder[]; // override dell'ordine
  embroideryPlacement: ... | null;
  handleGeometry: RenderHandleGeometry | null;
}

interface RenderSelection {
  fabricTextureUrl: string | null;
  fabricPatternScale: number;
  fabricRepeatMode: 'repeat'|'clamp'|'mirror';
  handlePreset: HandlePatternPreset | null;
  embroideryImageUrl: string | null;
  shadowsBoost?: number; // multiply ripetuto N volte
  highlightsBoost?: number;
}
```

8.2 Moduli del motore

File	LOC	Responsabilità
types.ts	230	Tipi e costanti (DEFAULT_LAYER_ORDER, STANDARD_FABRIC_ZONES).
assetLoader.ts	94	Cache di <code>HTMLImageElement</code> , <code>preloadSceneAssets</code> parallelo con <code>Promise.allSettled</code> .
layerComposer.ts	433	Orchestratore: ordina i livelli, ne pilota il disegno sul context, applica boost ombre/luci.
textureRenderer.ts	281	Disegno di una zona tessuto: maschera + pattern + transform + blend.
handlePath.ts	172	Parser/utility del documento path centerline manico.
handlePreset.ts	94	Caricamento e validazione preset righe del manico.
handleStripeRenderer.ts	586	Renderer procedurale strisce manico (la regola: bande longitudinali continue, colore = $f(U)$).
sidePartPathFallback.ts	118	Calcolo path di fallback per side parts senza centerline esplicita.
assetValidator.ts	137	Validazione preventiva: dimensioni, esistenza zone, sovrapposizioni.

8.3 Il flusso `composeScene()`

```
// pseudo-code semplificato
async function composeScene(canvas, scene) {
  const ctx = canvas.getContext('2d');
  canvas.width = scene.view.canvasWidth;
  canvas.height = scene.view.canvasHeight;
  ctx.clearRect(0, 0, canvas.width, canvas.height);
```



```
// 1) Preload TUTTI gli asset richiesti
const assets = await preloadSceneAssets(...);

// 2) Costruisci la pipeline ordinata di layer
const pipeline = buildPipeline(scene.view, scene.selection);

// 3) Per ogni layer, applica blend+opacity, disegna
for (const layer of pipeline) {
  ctx.save();
  ctx.globalCompositeOperation = layer.blendMode;
  ctx.globalAlpha = layer.opacity;
  layer.draw(ctx, assets);
  ctx.restore();
}

// 4) Eventuale boost ombre/luci (ripeti il blend N volte)
applyShadowsBoost(ctx, ...);
}
```

8.4 Cache asset

assetLoader.ts mantiene una `Map<url, HTMLImageElement>` globale con deduplica delle Promise in flight. Questo evita di scaricare due volte la stessa maschera quando l'utente cambia colore tessuto (l'unica cosa che cambia è il blend, gli asset sono gli stessi). La cache non ha eviction policy: in sessione lunga può crescere linearmente con il numero di modelli visitati. Si veda sezione 19 per la proposta di LRU bounded.

9. Pipeline di composizione livelli (Z-index)

La regola cardine è: **Base < Tessuto < Manici < Ricamo < Dettagli**. L'ordine di default è tabulato in DEFAULT_LAYER_ORDER e può essere overrideato per vista tramite layer_order_rules.

Z	layer_type	blend	opacity	Sorgente
0	base	normal	1.0	bag_views.base_image_url
10	fabric	normal	1.0	mask_zones (category=fabric) × selection.fabricTextureUrl
20	fabric_overlay	multiply	1.0	mask_zones.local_overlay_url
30	handle	normal	1.0	handleStripeRenderer (procedurale) o handle_geometries
40	handle_overlay	multiply	1.0	handle_geometries.shadow_url
50	embroidery	normal	1.0	embroideries × embroidery_placements
60	overlay_shadows	multiply	1.0	bag_views.overlay_shadows_url (boost ×N)
70	overlay_highlights	screen	0.8	bag_views.overlay_highlights_url (boost ×N)
80	overlay_details	normal	1.0	bag_views.overlay_details_url
—	overlay_zones	normal	1.0	mask_zones (category=overlay), es. zipper colorato

9.1 Blend modes

Il sistema usa l'API globalCompositeOperation nativa del Canvas 2D, che supporta i blend mode: multiply, screen, overlay, soft-light, hard-light, normal. La differenza chiave rispetto a Photoshop è che il Canvas 2D non implementa esattamente le formule di Photoshop per "overlay" e "soft-light"; per coerenza con i layer designer Photoshop si raccomanda di lavorare in solo multiply/screen e tradurre eventuali "overlay" come somma di multiply + screen separati.

9.2 Boost ombre/luci

Il parametro opzionale shadowsBoost ripete il blend multiply N volte sullo stesso overlay shadows, amplificando le ombre profonde quando il tessuto applicato è chiaro e l'overlay originale ha poca densità. Stesso meccanismo per highlightsBoost (screen ripetuto). Implementazione in layerComposer.ts tramite ciclo che usa canvas offscreen come buffer.

9.3 Rendering della singola zona tessuto

renderTexturedZone() in textureRenderer.ts implementa il pattern:

```
// 1) Disegna la maschera su un offscreen canvas
maskCtx.drawImage(maskImg, 0, 0);

// 2) Crea pattern dal tessuto con transform (scale, offset, rotation)
const pat = patternCtx.createPattern(textureImg, repeatMode);
pat.setTransform(matrixFromTransform(transform));
patternCtx.fillStyle = pat;
patternCtx.fillRect(0, 0, w, h);

// 3) Componi: pattern visibile SOLO dove la maschera è bianca
patternCtx.globalCompositeOperation = 'destination-in';
patternCtx.drawImage(maskCanvas, 0, 0);

// 4) Disegna il risultato sul canvas principale con opacity = shadingStrength
mainCtx.globalAlpha = zone.shadingStrength;
mainCtx.drawImage(patternCanvas, 0, 0);
```

10. Gestione tessuti e texture seamless

Il sistema gestisce due famiglie di texture:

10.1 Texture multi-colore (B/N + tinta multiply)

Per tessuti come neoprene, dove esistono molte varianti colore della stessa trama, si carica *una sola* texture grayscale seamless e si memorizzano N record in `fabric_colors` con hex diverso. Al runtime il client compone: texture B/N in normal + rettangolo pieno tinta in blend multiply, all'interno della stessa maschera. Implementato in `src/lib/textureMultiply.ts`.

10.2 Texture full-color

Per pelli, jacquard, stampe — la texture viene caricata già colorata e usata as-is senza tint multiply. `fabric_colors` in questo caso punta a `derived_fabric_id` (un altro fabric con `texture_url` proprio) per riusare il modello "una riga = una texture".

10.3 Generazione seamless via AI

La pipeline di generazione seamless da fotografia reale (sezione 6.1) ha sostituito un precedente approccio matematico/procedurale (autocorrelazione + cross-fade dei bordi) che produceva artefatti visibili (effetto "scacchiera" o "pois" sui pattern regolari). L'approccio attuale, basato su Gemini image preview, eredita le best practice di Patterned.ai: il prompt richiede continuità toroidale, lighting flat, conservazione della struttura fibrosa reale. Il file legacy `src/lib/textureSeamless.ts` contiene ancora utility procedurali deprecate, mantenute per fallback ma non più referenziate dal flusso principale.

10.4 Validazione e anteprima

`src/components/admin/TextureRepeatPreview.tsx` mostra la texture ripetuta 4×4 in tempo reale, permettendo all'admin di verificare la continuità prima di salvare. `TextureStudio.tsx` aggiunge controlli di scale/offset/rotation per il fine-tuning della transform per zona.

11. Gestione manici, path e strisce longitudinali

I manici sono l'elemento più complesso del rendering perché combinano: una geometria fissa (silhouette/ombre/luci PNG), un path centerline (polilinea di skeleton con tangenti) e un pattern di colori procedurale (preset di righe).

11.1 Modello dati manico

handles (catalogo tipi) → handle_geometries (1 record per coppia vista×manico, contiene mask_url, shadow_url, highlight_url, hardware_url, path_json, default_width). handle_side_parts aggiunge fettuccine laterali (loop_left, loop_right) con propri PNG. handle_pattern_presets definisce i pattern righe indipendentemente dalla borsa: un preset = un array di [colore, larghezza] longitudinali.

11.2 path_json — struttura del documento centerline

```
interface HandlePathDocument {
  version: 1;
  points: Array<{
    x: number; // pixel coords nel canvas vista
    y: number;
    tangent: { dx: number; dy: number }; // versore tangente normalizzato
    width: number; // larghezza locale del manico (px)
  }>;
  segments?: Array<{ from: number; to: number }>; // segmentazione
}
```

La centerline è ottenuta tramite **scheletrizzazione nearest-neighbor** sui pixel della maschera del manico (algoritmo iterativo di erosione morfologica), poi semplificata in ~30–80 punti, e per ciascun punto si calcola la tangente come media dei 2 vicini. La larghezza locale è la distanza dal nearest pixel di bordo. L'intera procedura è automatizzabile via /mask-tool, pagina interna che esegue l'analisi e fa upload diretto al DB.

11.3 Strisce longitudinali — regola sacra

■ **Nota:** Le strisce del manico sono **bande LONGITUDINALI continue**. Il colore è funzione solo della coordinata **U** (= ascissa curvilinea lungo la centerline), **costante in V** (= coordinata ortogonale). Mai blocchi, mai scacchiera, mai tile lungo la lunghezza. Questa regola è memorizzata in `mem://rendering/strisce-manico-longitudinali` ed è la causa principale di refactoring di `handleStripeRenderer.ts` nelle iterazioni precedenti.

11.4 Algoritmo handleStripeRenderer (sintesi)

```
// Per ogni pixel (px,py) all'interno della maschera manico:
// 1) trova il punto più vicino sulla centerline → indice i e distanza d
// 2) U = ascissa curvilinea cumulativa fino a points[i] + proiezione locale
// 3) V = d (distanza dalla centerline, già firmata vs tangente)
// 4) cerca la striscia s tale che U ∈ [stripe_s.start, stripe_s.end] (modulo lunghezza totale)
// 5) colore_pixel = stripe_s.color
// 6) se |V| > width_locale/2 → trasparente
// Output: ImageData che viene poi composta con shadow/highlight overlay.
```

Il feathering 1–2px sul bordo ($V \approx \text{width}/2$) è applicato con un alpha lineare per evitare jaggies. Performance: ~250ms per un manico full-HD (1762×1770), ottimizzabile con WebWorker (vedi sezione

19).

12. Maschere, feathering e overlay

12.1 Maschere — specifiche pittoriche

- PNG 32-bit RGBA, sfondo trasparente, colore di pittura bianco puro #FFFFFF.
- Feathering 1–3px sul bordo: morbido per evitare aliasing, non eccessivo per non far sbavare il tessuto fuori dalla zona.
- Mai includere hardware metallico (cerniere, fibbie, anelli) nella maschera tessuto: questi vanno in `overlay_details` separato.
- Convenzione di naming: `mask_body_front_main`, `mask_body_left_side`, `mask_body_top_band`, `mask_body_bottom_fold`. Il parser ZIP normalizza `alias legacy` (es. `mask_body_main` → `mask_body_front_main`).

12.2 Overlay — tipologie

- **overlay_shadows**: PNG grayscale densità ombre, blend multiply. Genera profondità.
- **overlay_highlights**: PNG grayscale luci speculari, blend screen, opacity 0.8. Genera lucentezza.
- **overlay_details**: PNG colore con alpha. Hardware fisso (cerniere, fibbie, etichette in pelle, cuciture chiare). Blend normal sopra tutto.
- **local_overlay** (per `mask_zone`): overlay locale alla singola zona (es. una piccola ombra nel centro del fronte). Multiply.

12.3 Estrazione overlay automatica

`src/lib/overlayExtractor.ts` implementa una pipeline di *estrazione automatica* dell'overlay shadows da una foto del prodotto reale: converte in HSL, estrae il canale luminanza, normalizza con curva tonale, restituisce un PNG grayscale + alpha. È un'utilità che velocizza la preparazione asset ma il risultato richiede sempre revisione artistica manuale.

13. Frontend e interfaccia utente

La SPA cliente è organizzata in due gerarchie: **flusso utente finale** (configuratore 6 step, `src/pages/Index.tsx` + `BagCanvas.tsx`) e **pannello admin** (catalogo, asset, layout, sezione 14).

13.1 Layout e responsive

La UI è desktop-first con breakpoint Tailwind `sm/md/lg`. Il canvas di preview occupa metà schermo a desktop, top-card a mobile. Il pannello laterale di selezione (tessuto, manico, ricamo) usa shadcn Tabs verticali a desktop, accordion a mobile. La paletta semantica HSL definita in `src/index.css` garantisce coerenza tra light e dark mode (anche se il dark mode non è ancora attivo).

13.2 Componenti chiave

- `BagCanvas.tsx` — Wrapper React attorno a `composeScene()`. Gestisce `resize`, `devicePixelRatio`, `debouncing` dei re-render quando lo stato `selection` cambia.
- `useRenderingData.ts` — Custom hook che fetcha view + maschere + selection-related entities da Supabase via `react-query`, restituisce un `RenderScene` pronto.
- `NavLink.tsx` — Wrapper di `react-router NavLink` con stili shadcn coerenti.
- `NotFound.tsx` — Fallback 404 con link a `/admin` se in dev mode.

13.3 Form e validazione

Tutti i form admin usano **react-hook-form + zod** per la validazione typesafe. Lo schema zod è co-locato con il componente (no schema centralizzato), pattern accettato in fase di prototipo ma da consolidare in `src/schemas/` centralizzato per deduplicare le regole condivise (es. validazione hex color, slug).

13.4 Notifiche e feedback

Sonner (toast moderno) per feedback istantanei (upload riuscito, errore di rete). Per dialog di conferma distruttiva (es. `delete bag_model`) si usa `AlertDialog` di shadcn. Loading skeletons via shadcn `Skeleton` durante i fetch `react-query`.

14. Backend amministrativo (Admin Panel)

Il pannello admin è raggiungibile sotto `/admin/*`. Allo stato attuale **non è protetto da autenticazione** in modalità preview, scelta da revertire prima del go-live. Il layout è gestito da `AdminLayout.tsx` che fornisce sidebar con voci:

- **Dashboard** (`/admin`) — Cards con count rapidi e shortcut.
- **Modelli borsa** (`/admin/models`) — CRUD modelli, editor viste annidato, link a upload ZIP per modello.
- **Tessuti** (`/admin/fabrics`) — CRUD tessuti, gestione `fabric_colors`, generazione seamless da foto, anteprima ripetizione.
- **Stili manici** (`/admin/handle-styles`) — Hub unificato a 4 tab: Tipi di manico, Corde, Texture reali, Pattern righe.
- **Carica file** (`/admin/upload`) — Pagina canonica di import ZIP, con targeting opzionale via query params `bagModelId/bagViewId`.
- **Texture Lab** (`/admin/texture-lab`) — Sandbox per testare seamless e enhance.
- **Mask Tool** (`/mask-tool`) — Tool interno automatizzato per analisi skeleton e upload handle path/preset.

14.1 Editor di vista (BagViewsEditor)

Il componente più complesso del pannello: per ogni vista di un modello permette di caricare base+overlays, gestire le `mask_zones` (creare, editare transform, override texture, toggle attive), configurare `layer_order_rules`, settare `embroidery_placement`. `FabricZonesEditor.tsx` e `OverlayZonesEditor.tsx` sono i sotto-editor specializzati, `MaskSlotUploader.tsx` gestisce l'upload di una singola maschera con anteprima sovrapposta sulla base image.

14.2 Strumenti di authoring avanzati

- **OverlayExtractorDialog** — Estrae overlay shadows da una foto del prodotto.
- **TextureOrientationDialog** — Imposta rotation di una texture per allinearla alla zona (utile per pattern direzionali).
- **HandleAlignmentPreview** — Sovrappone la centerline al PNG manico per validare il `path_json`.
- **HandlePresetEditor + HandlePresetPreview** — Editor visuale dei preset righe con preview real-time.

14.3 Pattern di accesso ai dati

Tutti i componenti admin usano react-query con queryKey stabili (`['bag_models']`, `['bag_views', modelId]`) e invalidate puntuale post-mutazione. Le mutazioni (`insert/update/delete`) usano `useMutation` con `optimistic update` solo per toggle booleani; le operazioni distruttive richiedono `refetch`.

15. Pipeline di import asset (ZIP canonico)

L'unica via di import asset borsa è /admin/upload. La logica risiede in src/lib/assetPack.ts (~950 LOC), il cuore della pipeline.

15.1 Struttura ZIP attesa

```
asset_pack.zip
■■■ asset_manifest.json # metadata: modelSlug, viewSlug, canvasW, canvasH, ...
■■■ original.png # foto sorgente (riferimento)
■■■ handle_presets.json # array di preset globali (opzionale)
■■■ README.txt
■■■ body/
■ ■■■ mask_body_front_main.png
■ ■■■ mask_body_left_side.png
■ ■■■ mask_body_right_side.png
■ ■■■ mask_body_top_band.png
■ ■■■ mask_body_bottom_fold.png
■ ■■■ overlay_body_shadows_multiply.png
■ ■■■ overlay_body_highlights_screen.png
■ ■■■ overlay_body_details.png
■■■ handle_geometry/
■ ■■■ mask_handle_main_full.png
■ ■■■ handle_path.json
■ ■■■ overlay_handle_shadows_multiply.png
■ ■■■ overlay_handle_highlights_screen.png
■ ■■■ mask_handle_side_loop_left.png
■ ■■■ mask_handle_side_loop_right.png
■ ■■■ overlay_handle_side_loop_*.png
■■■ previews/preview_01.png
■■■ references/reference_01.png
```

15.2 Step della pipeline (commitAssetPack)

- **Parse manifest** — Validazione zod del manifest. Error fail-fast con messaggio chiaro.
- **Normalizzazione alias** — Mappa nomi legacy ai nomi canonici (es. mask_body_main → mask_body_front_main).
- **Risoluzione target** — Se query params bagModelId/bagViewId presenti, ignora manifest e applica al target. Altrimenti upsert per (modelSlug, viewSlug).
- **Upload PNG** — Tutti i file body/* e handle_geometry/* vengono caricati in parallelo su configurator-assets con upsert.
- **Upsert DB** — bag_models, bag_views, mask_zones, handle_geometries, handle_side_parts, embroidery_placements, layer_order_rules vengono allineati. Operazione idempotente.
- **Handle presets globali** — handle_presets.json + manifest.handlePresets[] vengono dedupati per name e upsertati su handle_pattern_presets.
- **Side loops** — Auto-detect: se mask_handle_side_loop_*.png esistono, crea record handle_side_parts collegato.
- **Validazione finale** — assetValidator scarica e verifica dimensioni, restituisce report.

15.3 Targeting da contesto borsa

Da Admin → Modelli borsa l'admin può aprire l'import vincolato:

- Bottone «Carica file (ZIP)» sotto i campi della borsa → /admin/upload?bagModelId=... (forza la borsa, vista dal manifest).
- Bottone «ZIP» su ogni riga vista → /admin/upload?bagModelId=...&bagViewId=... (forza borsa + vista).
- Quando i query params sono presenti, un banner blu in cima alla pagina segnala il vincolo attivo, con link «Rimuovi vincolo».

16. Flusso utente (6 step) e integrazione Shopify

La UI cliente è organizzata in 6 step lineari con stato sincronizzato real-time sul canvas:

#	Step	Azione utente	Effetto sul canvas
1	Modello	Sceglie bag_model da galleria	Carica vista default + overlay base
2	Tessuto	Sceglie fabric + fabric_color	Applica texture in tutte le zone fabric_*
3	Manici	Sceglie handle + preset righe + colori	Renderizza handleStripe procedurale
4	Ricamo	Sceglie embroidery + posiziona	Disegna PNG ricamo con transform
5	Riepilogo	Verifica scelte + prezzo finale	Snapshot preview, lookup pricing_rules
6	Conferma	Click «Aggiungi al carrello»	INSERT in configurations + redirect Shopify

16.1 Aggiunta al carrello Shopify

Lo step 6 effettua due operazioni in sequenza:

```
// 1) Persisti la configurazione
const { data: cfg } = await supabase.from('configurations').insert({
  session_id: sessionId,
  bag_model_id, fabric_id, handle_id, embroidery_id,
  final_price, preview_url,
  shopify_product_id: bagModel.shopify_product_id,
  status: 'pending',
  expires_at: new Date(Date.now() + 7*86400000).toISOString(),
}).select().single();

// 2) Compose Line Item Properties e redirect Shopify
const props = {
  _configId: cfg.id,
  Modello: bagModel.name,
  Tessuto: `${fabric.name} (${color.hex})`,
  Manici: `${handle.name} preset:${preset.name}`,
  Ricamo: embroidery?.name ?? 'Nessuno',
  Anteprima: preview_url,
};
window.location.href =
`https://shop.example.com/cart/add?id=${variantId}&` +
Object.entries(props).map(([k,v]) =>
`properties[${k}]=${encodeURIComponent(v)}`).join('&');
```

16.2 Tracciabilità ordini

Lo _configId nel carrello permette al backend Shopify (via webhook order/created) di richiamare la configurazione completa dal DB Supabase per: ri-renderizzare il preview ad alta risoluzione, allegarlo alla bolla di produzione, gestire eventuali resi.

17. Memorizzazione utenti e gestione sessione

Allo stato attuale il configuratore **non richiede login** per la maggior parte delle interazioni. La sessione utente è identificata da `session_id`, UUID generato client-side e persistito in `localStorage` sotto chiave `configurator_session`. Tutte le configurazioni vengono salvate con quel `session_id`; al login opzionale (Supabase Auth email+password) viene aggiunto `user_id` al record per linkare le configurazioni storiche.

17.1 Tabella users e profili

Il progetto usa direttamente `auth.users` (gestita da Supabase) e **non** ha attualmente una tabella `profiles` dedicata. Quando si introdurranno ruoli admin la tabella `user_roles` dovrà essere creata (vedi sezione 5.1) e una `profiles` potrà ospitare `display_name`, `avatar`, lingua preferita.

17.2 Auth flow target

- Email + password con *email confirmation obbligatoria* (in fase admin di prod).
- Google OAuth come secondo provider (configurazione in supabase config + Google Cloud Console).
- Magic link via Supabase per gli admin (passwordless).
- Mai anonymous sign-up: ogni utente deve avere identità verificata.

17.3 Persistenza configurazione cross-device

Quando l'utente è loggato, le configurazioni sono recuperabili da qualunque dispositivo via query `SELECT * FROM configurations WHERE user_id = auth.uid() ORDER BY created_at DESC`. Per utenti anonimi la persistenza è solo locale (`session_id` in `localStorage`) e finestra 7 giorni (`expires_at`).

18. Funzionalità deboli, debiti tecnici, rischi produzione

■ **Nota:** Questa sezione è la più importante in ottica delivery. Elenca senza autocensura le aree fragili del progetto al momento della redazione, con livello di severità e impatto stimato.

18.1 Sicurezza (alto rischio)

- **RLS aperte** — Tutte le 58 policy ammettono lettura/scrittura pubblica. Bug-bounty: chiunque può cancellare l'intero catalogo via REST. *Severità: critica. Mitigazione: introdurre user_roles + has_role come da sezione 5.1 prima di qualunque ambiente non-dev.*
- **Admin panel non protetto** — /admin/* è raggiungibile da chiunque conosca l'URL. *Severità: alta. Mitigazione: route guard React + RLS lato DB.*
- **Edge functions pubbliche** — verify_jwt=false su seamless-from-photo permette drenaggio quota Lovable AI Gateway. *Severità: alta. Mitigazione: rate-limit IP via Deno KV o turbo+cookie.*
- **Bucket admin-assets pubblico** — Esposizione di ZIP che possono contenere foto sorgenti riservate. *Severità: media. Mitigazione: rendere privato + signed URL.*
- **Nessun audit log** — Le modifiche al catalogo non sono tracciate. *Severità: media. Mitigazione: tabella audit_log + trigger.*

18.2 Robustezza del codice

- src/lib/textureSeamless.ts (461 LOC) è codice deprecato non più referenziato dal flusso principale ma ancora compilato. *Da eliminare per ridurre bundle e confusione.*
- src/lib/assetPack.ts a 950 LOC è oltre la soglia di mantenibilità. Va spezzato in moduli (manifest, normalizer, uploader, dbWriter, validator).
- Nessun test di integrazione per composeScene(): solo un test placeholder in src/test/example.test.ts. *Rischio regressioni grafiche silenziose.*
- Mancano test E2E (Playwright) per il flusso 6-step. *Rischio rotture di percorso utente.*
- handleStripeRenderer.ts (586 LOC) ha logica complessa senza unit test e con commenti scarsi. *Bus-factor: 1.*
- Errori di rete (Storage 503, Edge timeout) sono catturati con toast generici, senza retry strategy. *Rischio degrado UX in connessioni instabili.*
- session_id in localStorage non è criptato: su browser condiviso si possono recuperare configurazioni altrui.

18.3 Operatività e dati

- **Nessun seeding automatico** — Un nuovo dev deve configurare manualmente bag_models, fabrics, handles. *Mitigazione: script SQL di seed in supabase/seed.sql.*
- **Manifest ZIP non versionato** — asset_manifest.json non ha campo schemaVersion: cambi futuri romperanno gli ZIP storici. *Mitigazione: introdurre versioning + migrazione asset.*

- **configurations.expires_at** non ha cron di cleanup. Tabella destinata a crescere indefinitamente.
- **Nessun backup off-site** dichiarato dei bucket Storage.

19. Performance, colli di bottiglia e ottimizzazioni

19.1 Misurazioni attuali

Operazione	Stato	Target	Hot path
Cold start configuratore	~2.8s	<1.5s	Bundle 450KB gz, react-query idle
Frame budget composeScene()	~120ms	<60ms	handleStripeRenderer + boost overlay
Cambio tessuto (con asset cached)	~80ms	<40ms	Re-disegno completo, no diff
Generazione seamless AI	~25-45s	<15s	Edge → Gemini image preview
Upload ZIP (50MB)	~30s	<15s	Sequenziale, no chunked upload

19.2 Strategie di ottimizzazione

- **WebWorker per handleStripeRenderer** — Spostare il pixel-loop su worker, ritornare ImageData via postMessage(transferable). Riduce blocking del main thread.
- **OffscreenCanvas + render dirty regions** — Mantenere buffer per layer e ridisegnare solo i layer che cambiano (es. cambio colore tessuto invalida solo fabric+overlay).
- **Image cache LRU bounded** — Sostituire la Map illimitata di assetLoader con LRU(50) per evitare memory leak in sessione lunga.
- **Code splitting admin** — Le route /admin/* sono lazy-loadable: l'utente finale non scarica i ~3000 LOC di admin UI.
- **Image preloading anticipato** — Quando l'utente è allo step 1, fare prefetch a bassa priorità degli asset dei tessuti più popolari (analytics-driven).
- **Cache HTTP CDN** — I PNG catalogo sono immutabili (filename versioned). Settare Cache-Control: public, max-age=31536000, immutable.
- **Chunked upload ZIP** — Usare resumable.js o tus per upload affidabili di ZIP >50MB.

19.3 Bundle analysis

Il bundle attuale è dominato da: shadcn-ui+Radix (~120KB gz), react+react-dom (~45KB), @supabase/supabase-js (~38KB), jszip (~28KB), @tanstack/react-query (~18KB). Eliminare componenti shadcn non usati (audit con knip o ts-prune) può recuperare 30-40KB.

20. Stato dell'arte grafico e proposte di miglioramento

Il risultato grafico attuale è funzionalmente corretto ma **esteticamente ancora limitato**: il composito finale ha ancora un'aria da mockup tecnico più che da rendering fotorealistico luxury. Le cause principali sono identificabili e affrontabili in modo incrementale.

20.1 Cause della limitazione attuale

- **Overlay shadows/highlights generici** — Spesso prodotti via overlayExtractor automatico, mancano di nitidezza e densità non uniforme.
- **Texture seamless con risoluzione bassa** — Tile a 512px o 1024px che, ripetuti su zone grandi, mostrano pattern visibili. Servirebbero 2048+ con dettagli micro-tessili.
- **Mancanza di micro-shading per zona** — Le zone fabric sono tinte unite shaded solo dall'overlay globale; non ci sono ombre cucite ai bordi delle cuciture.
- **Hardware (cerniere, anelli) statico** — overlay_details unico per vista, niente reflection adattiva al colore tessuto.
- **Manico procedurale piatto** — Le strisce longitudinali del manico non hanno volume; l'overlay shadow del manico è generico e non si adatta al pattern righe.
- **Assenza di environment lighting** — Tutto il rendering ipotizza luce ambient diffusa fissa. Cambiando background colore (HTML), la borsa non si adatta.

20.2 Proposte di miglioramento (priorità decrescente)

- **1. Pipeline asset 2.0 con normal map** — Affiancare a base_color un PNG normal map per ogni vista. Combinare in shader WebGL/WebGPU per relighting dinamico. Investimento alto, salto qualitativo enorme.
- **2. Multi-resolution texture (mipmap)** — Caricare 3 livelli di seamless (512/1024/2048) e selezionare in base alla scala di display. Riduce moiré e migliora dettaglio.
- **3. Cuciture procedurali** — Calcolare automaticamente le linee di cucitura sui bordi delle mask_zones (offset 4px verso l'interno) e disegnarle con un pattern stitch. Aumenta drasticamente il fotorealismo.
- **4. Hardware multivista** — overlay_details parametrizzato per colore tessuto: ad esempio cerniere nichel su tessuti chiari, ottone su scuri.
- **5. Shading adattivo manico** — Generare overlay shadow del manico in funzione del preset righe (le strisce scure ricevono luce diversa).
- **6. Anti-aliasing supersampling** — Disegnare il canvas a 2× la risoluzione e downscale con filtro Lanczos. Riduce jaggies sui bordi maschere.
- **7. Studio lighting presets** — 3 preset di illuminazione (showroom, daylight, indoor warm) selezionabili dall'utente, applicati come tone-curves sul composito finale.

20.3 Procedura di onboarding asset — proposta riformulazione

La procedura attuale (ZIP canonico con naming rigido) è funzionale ma poco accessibile. Proposta UX:

- **Wizard multi-step in 5 fasi** — (1) carica foto sorgente, (2) il sistema propone overlay shadow estratto + permette refinement con pennello, (3) carica/disegna maschere zona con tool di brush integrato (no Photoshop esterno), (4) carica handle PNG e il sistema calcola path automaticamente con preview, (5) preview finale + commit.
- **Editor maschere in-browser** — Tool brush+eraser con modello base sotto, salvataggio diretto come PNG con feathering automatico. Elimina la dipendenza da Photoshop per molti casi.
- **AI proposta automatica di mask zones** — Modello segmentation (SAM via API) che propone le zone tessuto plausibili dato il PNG sorgente. Admin valida e raffina.
- **Preview live ZIP** — Caricato il file, prima del commit DB, mostrare il rendering completo con tessuti placeholder. Permette di scartare ZIP malformati prima di sporcare il catalogo.
- **Versionamento asset** — Ogni vista mantiene history degli asset caricati con possibilità di rollback. Sicurezza operativa.

21. Roadmap di evoluzione e raccomandazioni finali

21.1 Sprint 1 — Hardening sicurezza (1–2 settimane)

- Introdurre `user_roles` + `has_role` + policy ristrette su tutto il catalogo.
- Proteggere `/admin/*` con route guard + redirect a login.
- Rendere admin-assets bucket privato + signed URL.
- Rate-limit edge functions via Deno KV.
- Audit log su mutations critiche.

21.2 Sprint 2 — Refactor codebase (2 settimane)

- Eliminare `textureSeamless.ts` deprecato.
- Spezzare `assetPack.ts` in 5 moduli (~200 LOC ciascuno).
- Test unitari su `composeScene()` con fixture asset.
- Test E2E Playwright sul flusso 6-step.
- Bundle audit + rimozione componenti `shadcn` non usati.

21.3 Sprint 3 — Performance (1 settimana)

- `WebWorker` per `handleStripeRenderer`.
- `OffscreenCanvas` + dirty regions in `layerComposer`.
- LRU cache in `assetLoader` (max 50 immagini).
- Code splitting per `/admin`.
- Cache-Control immutable per asset versionati.

21.4 Sprint 4 — Pipeline asset 2.0 (3–4 settimane)

- Wizard di onboarding con AI segmentation.
- Editor maschere in-browser.
- Versionamento asset per vista.
- Preview live pre-commit ZIP.
- Mipmap multi-resolution per texture.

21.5 Sprint 5 — Salto fotorealistico (4–6 settimane)

- Migrazione del composer a WebGL2 con shader fragment per relighting.
- Normal map per asset chiave.

- Cuciture procedurali generate da contorno maschere.
- Hardware adattivo per tonalità tessuto.
- Studio lighting presets selezionabili dall'utente.

21.6 Considerazioni finali

Il progetto è in stato di **solida fondazione tecnica** con un'architettura pulita e ben separata. Le criticità individuate sono tutte affrontabili senza riscrivere l'architettura: il piano in 5 sprint sopra delineato porta il sistema da prototipo funzionante a prodotto production-ready in circa **3 mesi di sviluppo focalizzato**. Il salto qualitativo grafico (sprint 5) richiede competenze WebGL/shading e va affidato a profilo specialistico. L'aspetto più urgente prima di qualunque go-live resta la chiusura delle policy RLS e la protezione del pannello amministrativo.

Appendice A — Glossario tecnico

Termine	Definizione
Asset Pack	ZIP unico contenente tutti i PNG e JSON per importare una vista borsa nel sistema.
Blend mode	Modalità di composizione di due livelli (multiply, screen, normal). Implementata via <code>globalCompositeOperation</code> .
Centerline	Polilinea di skeleton del manico, sequenza di punti con tangente e larghezza locale.
Composer	Modulo <code>src/engine/layerComposer.ts</code> che orchestra il disegno dei livelli.
Edge Function	Funzione TypeScript Deno deployata su Lovable Cloud, esposta via HTTP POST.
Feathering	Sfumatura dei bordi di una maschera (1–3px) per evitare aliasing.
Idempotenza	Proprietà per cui ripetere un'operazione produce lo stesso risultato del farla una volta sola.
Line Item Properties	Metadati custom per riga di carrello Shopify, usati per trasportare la configurazione.
Mask zone	Area pittabile definita da una maschera bianco-su-trasparente, con una propria texture e blend.
Multiply	Blend mode che moltiplica i canali RGB; utile per ombre realistiche.
Overlay (asset)	PNG aggiuntivo applicato sopra la base con blend specifico (<code>shadows=multiply</code> , <code>highlights=screen</code> , <code>details=normal</code>).
Path JSON	Documento che descrive la centerline di un manico (<code>handle_geometries.path_json</code>).
Preset righe	Configurazione salvata di pattern righe longitudinali per manico (<code>handle_pattern_presets</code>).
RLS	Row Level Security PostgreSQL: regole dichiarative che filtrano l'accesso alle righe per utente.
Screen	Blend mode inverso del multiply, schiarisce; usato per highlights.
Seamless tile	Texture progettata per essere ripetuta in toroide senza giunzioni visibili.
SECURITY DEFINER	Modificatore di funzione PostgreSQL: la funzione gira coi privilegi del proprietario, non del chiamante.
Shadcn/ui	Libreria React di componenti basata su Radix UI primitives, distribuita come codice copy-paste.
Shading strength	Coefficiente 0–1 che modula quanto l'ombra base traspare attraverso il tessuto applicato.
Side part (manico)	Fettuccina laterale del manico vicino agli anelli, con propri PNG.
Strisce longitudinali	Bande continue lungo l'ascissa curvilinea del manico; colore = $f(U)$ costante in V .
Toroidale	Continuo sia in orizzontale sia in verticale (come la superficie di un toro). Riferito a tile seamless.
Z-index	Ordinamento di disegno dei livelli: indici bassi sotto, alti sopra.

Appendice B — Diagramma ER sintetico

Rappresentazione testuale delle relazioni principali tra le tabelle del catalogo:

```
bag_models (1) ■■< bag_views (N)
■
■■■■< mask_zones (N) [zone tessuto/handle/overlay]
■■■■< layer_order_rules (N) [override z-index/blend]
■■■■< handle_geometries (N) ■■< handle_side_parts (N)
■■■■< embroidery_placements (1)

bag_models (1) ■■< compatibility_rules (N) >■■ fabrics (1)
bag_models (1) ■■< compatibility_rules (N) >■■ handles (1)
bag_models (1) ■■< pricing_rules (N) >■■ fabrics (1)

fabrics (1) ■■< fabric_colors (N)
■■ derived_fabric_id ■■> fabrics (self, opzionale)

handles (1) ■■< handle_colors (N)
handles (1) ■■< cord_handle_compatibility (N) >■■ cord_collection (1)

handle_pattern_presets (N) [globali, non legati a singolo handle]

embroideries (N) ■■< embroidery_placements (1 per vista)

configurations (N):
■■ FK: bag_model_id, fabric_id, handle_id, embroidery_id
■■ session_id (utente anonimo) | user_id (auth.users) — uno dei due
```

Convenzioni chiave

- Tutte le PK sono UUID (gen_random_uuid()).
- Tutte le tabelle hanno created_at e updated_at gestiti da trigger.
- ON DELETE CASCADE su bag_views→mask_zones, bag_views→handle_geometries→handle_side_parts.
- Tabelle catalogo (fabrics, handles, embroideries, cord_collection) hanno is_active e sort_order per gestione UI.
- I prezzi vivono ESCLUSIVAMENTE in pricing_rules (modello+tessuto). bag_models.base_price è prezzo di partenza informativo.

— Fine del documento. Versione 1.0, redatto Maggio 2026.